

Super



<https://github.com/learn-co-curriculum/python-p3-super>



<https://github.com/learn-co-curriculum/python-p3-super/issues/new>

Learning Goals

- Reduce repeated code and enhance objects using inheritance.
 - Use `super` to call methods on a parent class from the child class.
 - Accomplish complex programming tasks using knowledge from previous modules.
-

Key Vocab

- **Inheritance**: a tool that allows us to recycle code by creating a class that "inherits" the attributes and methods of a parent class.
 - **Composition**: a tool that enables you to recycle code by adding objects to other objects. Rather than building on a base class as in inheritance, composition leverages the attributes and methods of an instance of another class.
 - **Subclass**: a class that inherits from another class. Colloquially called a "child" class.
 - **Superclass**: a class that is inherited by another class. Colloquially called a "parent" class.
 - **Child**: another name for a subclass.
 - **Parent**: another name for a superclass.
 - `super()` : a built-in Python function that allows us to manipulate the attributes and methods of a superclass from the body of its subclass.
 - **Decorator**: syntax that allows us to add functionality to an object without modifying its structure.
-

Introduction

So far, we've seen the benefits of using inheritance to create a group of classes that share certain characteristics and behaviors. However, up until now, the implementation of shared characteristics has been somewhat rigid. If class `Student` inherits from class `User`, we can choose to

either allow the `Student` class to inherit a certain method from `User` or overwrite that method with another implementation that is specific to `Student`.

But what if there is a method in the parent class that we want our child to share *some* of the functionality of? Or what if we want our child class to inherit a method from the parent and then augment it in some way? We can achieve this with the use of the `super` keyword.

Using `super` to Supercharge Inheritance

Let's say we are working on an education app in which users are either students or teachers. We have a parent, `User` class, that both our `Student` and `Teacher` classes inherit from.

Our `User` class has a method, `log_in()`, that sets an instance variable, `self.logged_in` equal to `True`.

```
class User:
    def log_in(self):
        self.logged_in = True
```

However, when a student logs into our app, we need to not only set their logged in attribute to `True`, we need to set their "in class" attribute to `True` as well. We could simply edit the `log_in()` method in the `User` class to account for this. But that doesn't make sense here. Remember that both `Student` and `Teacher` inherit from `User`. Teachers don't need to indicate that they are "in class", so we don't want to alter the `log_in()` method of our parent class and inadvertently give teachers some behavior that they don't want or need.

Instead, we can augment, or supercharge, the `log_in()` method *inside* of the `Student` class.

Let's take a look:

```
class Student(User):
    def log_in(self):
        super().log_in()
        self.in_class = True
```

Here, we re-define the `log_in()` method and tell it to inherit any functionality of the `log_in()` method defined in the parent, or "superclass," which is `User`.

In the `log_in()` method above, the `super` keyword will **call on the `log_in()` method as defined in the superclass**. Then, the additional code that we're adding into our `Student.log_in()` method will also run. We have therefore supercharged our `log_in()` method, for the `Student` class only.

You can see for yourself by adding a `print()` statement in the `log_in()` methods for both the `User` and `Student` classes. Run this code in the Python shell or in a new Python file:

```
class User:
    def log_in(self):
        print("User.log_in() called.")
        self.logged_in = True

class Student(User):
    def log_in(self):
        print("Student.log_in() called.")
        super().log_in()
        self.in_class = True

oneil = Student()
oneil.log_in()
# "Student.log_in() called."
# "User.log_in() called."
# True
```

As you can see by the output of running `log_in()` on a `Student` instance, the `super` keyword calls the specified method in the parent class, adding the functionality of the original method to your new method in a single line.

► *All this talk about superclasses! What do we call the class that inherits from a superclass?*

Calling `super` With Arguments

The `log_in()` method defined above didn't take any arguments, but often, we'll need to call methods and provide some data as arguments. For example, let's modify our `User` class to give it a `name` attribute:

```
class User:
    def __init__(self, name):
        self.name = name

    def log_in(self):
        self.logged_in = True
```

Let's also modify our `Student` class so that it can be initialized with a `name` and a `grade` attribute:

```
class Student(User):
    def __init__(self, name, grade):
        self.name = name
        self.grade = grade

    def log_in(self):
        super().log_in()
        self.in_class = True
```

While we can still create new students with our updated class definition, it's not particularly DRY — both the `Student` and `User` classes define a `name` instance variable and set it when the class is instantiated.

Ideally, we'd like to be able to call the `__init__` method on the `User` class when a new student is created.

`super` lets us do just that! We can call `super` with an argument from the `Student.__init__` method, which will call the `User.__init__` method with that argument. Try updating the example code like so:

```
class User:
    def __init__(self, name):
        print("User.__init__ called.")
        self.name = name

    def log_in(self):
        self.logged_in = True

class Student(User):
    def __init__(self, name, grade):
        print("Student.__init__ called.")
        super().__init__(name)
        self.grade = grade

    def log_in(self):
        super().log_in()
        self.in_class = True

oneil = Student("O'neil", 10)
# Student.__init__ called.
# User.__init__ called.
oneil.__dict__
# {'name': "O'neil", 'grade': 10}
```

Just like in the previous example, `super` is used to call a method on the superclass from the subclass — the only difference is that this time we are passing in arguments that are required by the method defined in the superclass.

► `__init__` is the method that uses `super()` the most. Why do you think that is?

Conclusion

Often when you are dealing with code that involves inheritance, you'll need to augment the functionality defined in the superclass with some additional behavior in the subclass. The `super` keyword allows you to do just that!

Resources

- [Python 3.8 Documentation](https://docs.python.org/3.8/) ➞ (https://docs.python.org/3.8/)
- [Inheritance - Python](https://docs.python.org/3/tutorial/classes.html#inheritance) ➞ (https://docs.python.org/3/tutorial/classes.html#inheritance)
- [Supercharge Your Classes With Python super\(\) - Real Python](https://realpython.com/python-super/) ➞ (https://realpython.com/python-super/)